Complaint Classifier Project — Complete Summary (Data → Cloud)

Everything built, explained, and debugged, from start to a live public API. Written to be re-readable later without needing to remember the conversation.

---

1. The goal

Build a small tool that reads a real customer complaint (about a bank, credit card, etc.) and automatically figures out which category it belongs to — using an LLM (Claude), wrapped in a web API, packaged with Docker, and deployed live to the cloud. This targets an internship's stated requirements: LLM experience, cloud/MLOps familiarity, Python skills.

Final result: a working, public API at `https://complaint-classifier-315192541584.us-central1.run.app/docs` — reachable by anyone, running independently of your laptop.

---

2. The data: CFPB Consumer Complaint Database

- **CFPB** = Consumer Financial Protection Bureau — a US government agency, publishes a public database of real financial complaints.
- **Issue hit:** as of **August 14, 2026**, CFPB stopped publishing the actual complaint text ("narrative") in this database — a real, recent policy change, not something done wrong.
- **Fix:** used a Kaggle dataset ("Bank Customer Complaint Analysis") built from CFPB data *before* the narrative field was removed.
  - **162,421 rows**, columns: `narrative` (complaint text) and `product` (true category)
  - Five categories: `credit_reporting`, `debt_collection`, `mortgages_and_loans`, `credit_card`, `retail_banking`

What a credit report is: a record (kept by Equifax/Experian/TransUnion) of your borrowing history — cards, loans, missed payments. Errors on it are `credit_reporting` complaints. This overlaps heavily with `debt_collection`, since a wrong report can trigger a debt collector to contact you — both topics genuinely blend together in real complaint text.

---

3. Key concepts explained (Q&A recap)

pandas — Python library that turns a CSV into a `DataFrame` (a queryable table).

API key — a secret string tied to your Anthropic account; every request to Claude includes it, like a "sticky note" proving who's asking / whose credit to charge. Never share it publicly.

HTTP request — "a message sent over the internet from one computer to another." Same mechanism as loading any website.

Environment variable (`~/.zshrc`) — a hidden Mac settings file, read automatically by every new Terminal window. Storing the API key here (instead of in code) keeps it out of any code you might share.

- `echo 'export KEY="..."' >> ~/.zshrc` → writes the setting into the file
- `source ~/.zshrc` → the word `source` is what forces Terminal to re-read the file *immediately*, in the currently open window (normally it's only read when a *new* window opens)

API (`api.py`) — turns a script from "runs once, prints, exits" into "stays open, listens for requests, answers on demand." Built with **FastAPI** (defines the service) + **uvicorn** (runs it, listens for requests).

Docker image vs. container:

- **Image** = the frozen blueprint/recipe result (built once via `docker build`)
- **Container** = a live, running instance of that image (started via `docker run`)
- **Dockerfile** = the plain-text recipe card that `docker build` follows to produce the image. Not the image itself — just instructions.

Why package with Docker at all, if it still just runs on your computer locally? Running it locally in Docker doesn't add value *by itself* — the value is that it proves the packaging works correctly, *before* handing the exact same package to a cloud service. Local `python api.py` only works because your laptop happens to have the right setup; a Docker image carries everything it needs, so it behaves identically anywhere — your laptop, a coworker's machine, or a cloud server.

Ports, explained:

- **Port 8000** — arbitrary, chosen when writing `--port 8000`. Only matters on your own machine for your own testing.
- **Port 443** — not arbitrary; it's the universal standard port for secure web traffic (HTTPS). Every secure site/API uses it. Your `anthropic` library uses it automatically.
- `127.0.0.1` — always means "this exact computer." Nobody else can ever reach this address, no matter whose computer types it.
- `0.0.0.0` (used inside Docker/Cloud Run) — means "accept requests from anywhere," not a specific address you visit.
- `-p 8000:8000` in `docker run` — a **port mapping**: "port 8000 on my laptop → port 8000 inside the container." The two numbers don't have to match; they're two separate ports that happen to share a name here. Without this flag, the container runs fine internally, but nothing outside it — not even your own browser — could reach it.

What "the cloud runs your image" actually means: you upload the finished image to a registry (cloud storage for images). The cloud service does the equivalent of `docker run` on a server you'll never see directly, assigns it a real public address, and keeps it alive — restarting it if it crashes. You never tell them "install Python 3.13, then pandas..." — the image already contains everything, so their job shrinks to "start this and keep it running."

---

4. The full request path (local, no Docker)

```
Your browser
   → GET 127.0.0.1:8000/classify
Your laptop (uvicorn server)
   → runs classify(), builds a prompt
   → HTTPS request to api.anthropic.com:443
Anthropic's servers
   → Claude reads prompt, picks a category
   → sends answer back over HTTPS
Your laptop (uvicorn server)
   → wraps answer as JSON: {"category": "..."}
Your browser
   → displays the result
```

Your laptop plays **two roles at once**: server (answering your browser) and client (calling Claude). It's the middleman.

5. The full request path (with Docker)

Same as above, but with one extra hop inserted at the very start/end:

```
Your browser → 127.0.0.1:8000
   → Docker port mapping (-p 8000:8000) forwards traffic
   → Container's uvicorn server (listening on 0.0.0.0:8000 inside)
   → [same trip to Anthropic on port 443 as above]
   → Container wraps JSON response
   → Forwarded back out through the port mapping
   → Your browser displays result
```

The Anthropic leg is **completely unaffected** by Docker — only the path *to your own code* changes.

6. The full request path (deployed on Cloud Run)

Identical again, except step 1 changes: instead of `127.0.0.1:8000` (only reachable by you), the request goes to a real public address (`https://complaint-classifier-....run.app`), reachable by anyone, hosted on Google's server instead of your laptop. The Anthropic leg stays exactly the same.

---

7. The code, with comments

`explore.py` — first look at the data

python

```python
import pandas as pd

# Load the CSV into a table (DataFrame)
df = pd.read_csv("complaints.csv")

# Drop rows with no complaint text
df = df.dropna(subset=["narrative"])

print("Number of usable complaints:", len(df))
print(df[["product", "narrative"]].head())
print(df["product"].value_counts())  # how many complaints per category
```

`classify.py` — test Claude's classification accuracy

python

```python
import os
import anthropic
import pandas as pd

# Connect to Claude using the API key saved in ~/.zshrc
client = anthropic.Anthropic(api_key=os.environ["ANTHROPIC_API_KEY"])

df = pd.read_csv("complaints.csv")

# 30 random rows. random_state=42 = same 30 rows every run (42 has no spe
sample = df.sample(30, random_state=42)

correct = 0
total = 0

# _ = row's index number (discarded, unused). row = the actual row data.
for _, row in sample.iterrows():
    narrative = row["narrative"]   # complaint text
    true_label = row["product"]    # correct answer

    # {narrative} gets replaced with the real text at this spot
    prompt = f"""Classify this consumer complaint into exactly one of the
credit_reporting, debt_collection, mortgages_and_loans, credit_card, reta

Complaint: {narrative}
```

```
Respond with only the category name, nothing else."""

    response = client.messages.create(
        model="claude-sonnet-4-5",
        max_tokens=20,
        messages=[{"role": "user", "content": prompt}]
    )

    # response.content[0].text = Claude's actual reply, nested inside the
    predicted = response.content[0].text.strip()
    match = predicted == true_label

    total += 1
    if match:
        correct += 1

    print(f"True label:      {true_label}")
    print(f"Claude predicted: {predicted}")
    print(f"Match: {match}")
    print("---")

accuracy = correct / total * 100
print(f"\nFinal accuracy: {correct}/{total} = {accuracy:.1f}%")
```

Result: 23/30 = 76.7% (~80% adjusted — one "miss" was actually a correct answer plus an unwanted extra sentence, which broke the exact-match check). Real mistakes clustered around `credit_reporting` vs `debt_collection` — a genuinely ambiguous pair in the data itself, not a model failure.

`api.py` — turn the script into an always-on service

```
python
```

```python
import os
import anthropic
from fastapi import FastAPI

app = FastAPI()  # the running service itself
client = anthropic.Anthropic(api_key=os.environ["ANTHROPIC_API_KEY"])

# This "door" opens at /classify. Visitors must provide a `text` value.
@app.get("/classify")
def classify(text: str):
    prompt = f"""Classify this consumer complaint into exactly one of the
credit_reporting, debt_collection, mortgages_and_loans, credit_card, reta

Complaint: {text}

Respond with only the category name, nothing else."""

    response = client.messages.create(
        model="claude-sonnet-4-5",
        max_tokens=20,
        messages=[{"role": "user", "content": prompt}]
    )

    predicted = response.content[0].text.strip()

    # return (not print) sends this back as the actual API response,
    # auto-converted to JSON: {"category": "credit_card"}
    return {"category": predicted}
```

Run locally: `uvicorn api:app --reload` → test at `http://127.0.0.1:8000/docs`

`Dockerfile` — package the whole project, FINAL working version

dockerfile

```
# Start from a small pre-made environment with Python 3.13
# (must match your local Python version — see Issue #2 below)
FROM python:3.13-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

# ${PORT:-8000} = use whatever port Cloud Run assigns via the PORT
# environment variable; fall back to 8000 if it's not set (e.g. local tes
# "CMD exec ..." (not CMD [...]) is required for ${PORT} to actually subs
CMD exec uvicorn api:app --host 0.0.0.0 --port ${PORT:-8000}
```

`requirements.txt`

```
bash

pip3 freeze > requirements.txt
```

Auto-generated list of every installed library + exact version, so Docker can recreate the same setup anywhere.

---

8. Every issue hit, and how it was fixed

Issue 1 — CFPB narrative field missing entirely

Symptom: downloaded CSV had 15 columns, no `narrative`/complaint text column. **Cause:** CFPB stopped publishing complaint narratives publicly on Aug 14, 2026 — a real policy change, not a download mistake. **Fix:** switched to a Kaggle dataset built from CFPB data before the removal.

Issue 2 — `TabError: inconsistent use of tabs and spaces`

Symptom: `classify.py` failed to run after pasting code into `nano`. **Cause:** mixed tabs and spaces in indentation (invisible on screen, but Python treats them as different). **Fix:** rewrote the file using `cat > file.py << 'EOF' ... EOF` (a heredoc) instead of manually indenting in `nano`, which avoids the ambiguity entirely.

Issue 3 — Claude occasionally broke the "respond with only X" instruction

Symptom: one response included Claude's correct answer plus an unrequested explanatory note, causing an exact-match check to incorrectly count it as wrong. **Cause:** LLM instructions are strong suggestions, not hard-enforced rules; the model sometimes adds clarifying text especially on ambiguous cases. **Not "fixed" per se** —

noted as a known, common LLM behavior. Real solutions (not yet implemented) would be stricter prompting or requesting structured (JSON) output.

Issue 4 — Docker build failed: `numpy==2.5.3` not found

Symptom: `docker build` failed while installing requirements. **Cause:** Dockerfile said `FROM python:3.11-slim`, but the local Mac (and therefore `requirements.txt`) used Python 3.13. `numpy==2.5.3` only exists for Python 3.13+, so it wasn't available under 3.11. **Fix:** changed the Dockerfile to `FROM python:3.13-slim` to match the local Python version.

Issue 5 — Cloud Run rejected the image: architecture mismatch

Symptom: `gcloud run deploy` failed with: *"Container manifest type ... must support amd64/linux."* **Cause:** the Mac is Apple Silicon (arm64 architecture); `docker build` defaults to building for the machine's own architecture. Cloud Run's servers run on amd64, a different architecture — an arm64 image simply cannot run there. **Fix:** rebuilt specifically targeting amd64:

```bash
docker buildx build --platform linux/amd64 -t <full-image-address> --push
```

Issue 6 — Cloud Run deploy failed: container didn't listen on the right port

Symptom: *"container failed to start and listen on the port defined by PORT=8080 ... within the allocated timeout."* **Cause:** the Dockerfile hardcoded `--port 8000`, but Cloud Run expects the container to listen on whatever port it specifies via a `PORT` environment variable (defaults to 8080). The container was listening on the wrong port, so Cloud Run's health check never got a response. **Fix:** changed the Dockerfile's last line to read the `PORT` variable dynamically, with a local fallback:

```dockerfile
CMD exec uvicorn api:app --host 0.0.0.0 --port ${PORT:-8000}
```

(Also required switching from bracket-style `CMD [...]` to shell-style `CMD exec ...`, since only the shell form processes `${PORT}` substitution.)

Issue 7 — Live URL appeared "stuck connecting" in browser

Symptom: visiting the live `/docs` URL seemed to hang. **Cause:** almost certainly a **cold start** — Cloud Run spins containers down when idle and takes 10-30+ seconds to spin back up on the first request; occasionally the browser tab's loading state gets confused by this delay. **Fix:** confirmed via `gcloud run services logs read` that the server was actually returning `200 OK` successfully multiple times — the server was fine. Opening a fresh browser tab resolved the stuck display.

---

9. Full deployment command reference (in order)

```bash
```

```
# 1. Install Docker Desktop, then confirm:
docker --version

# 2. Write requirements.txt
pip3 freeze > requirements.txt

# 3. Write the Dockerfile (see final version above)

# 4. Build & test locally
docker build -t complaint-classifier .
docker run -p 8000:8000 --env ANTHROPIC_API_KEY=$ANTHROPIC_API_KEY compla
# test at http://127.0.0.1:8000/docs

# 5. Install gcloud CLI
brew install google-cloud-sdk
echo 'export PATH=/opt/homebrew/share/google-cloud-sdk/bin:"$PATH"' >> ~/
source ~/.zshrc

# 6. Log in and set up the project
gcloud auth login
gcloud projects create complaint-classifier-app --name="Complaint Classif
gcloud config set project complaint-classifier-app
# (link a billing account in the browser console — required even for free
gcloud services enable run.googleapis.com artifactregistry.googleapis.com

# 7. Create a registry, authenticate Docker to it
gcloud artifacts repositories create complaint-classifier-repo \
  --repository-format=docker --location=us-central1
gcloud auth configure-docker us-central1-docker.pkg.dev

# 8. Build for the RIGHT architecture (amd64) and push in one step
docker buildx build --platform linux/amd64 \
  -t us-central1-docker.pkg.dev/complaint-classifier-app/complaint-classi
  --push .

# 9. Deploy to Cloud Run
gcloud run deploy complaint-classifier \
  --image=us-central1-docker.pkg.dev/complaint-classifier-app/complaint-c
  --platform=managed \
  --region=us-central1 \
  --allow-unauthenticated \
  --set-env-vars=ANTHROPIC_API_KEY=$ANTHROPIC_API_KEY

# 10. Check logs if something's wrong
gcloud run services logs read complaint-classifier --region=us-central1 -
```

Live result: `https://complaint-classifier-315192541584.us-central1.run.app/docs`

---

10. Progress checklist

Step	Status
Real labeled data sourced (worked around live CFPB policy change)	✅ Done
LLM classification working, accuracy measured (~77-80%)	✅ Done
Wrapped in a working API (FastAPI)	✅ Done
Packaged in Docker (fixed Python version mismatch)	✅ Done
Deployed live to Google Cloud Run (fixed architecture + port issues)	✅ Done
GitHub Actions auto-deploy on push	⬜ Not started
Logging (speed, cost, failure rate)	⬜ Not started
Formal, larger evaluation test set	⬜ Optional / partially done
Simple front-end interface (beyond `/docs`)	⬜ Not started

---

11. Cost note

Every Claude API call in this project has cost a tiny fraction of a cent — total spend across all testing is likely still well under $1 of the $5 credit added. Google Cloud's $300 free trial credit is also untouched beyond this small deployment, which itself falls well within Cloud Run's separate "always free" monthly quota. Check usage anytime at console.anthropic.com (Claude) and console.cloud.google.com/billing (Google).